

Lab 12

Parser combinators in Haskell - Part 2

Goals

In this lab you will learn to:

1. Write monadic parser combinators
2. Write a parser for JSON

Resources

Table 12.1: Lab Resources

Resource	Link
Functional parsing	https://youtu.be/dDtZLm7HIJs
Understanding parser combinators	https://fsharpforfunandprofit.com/posts/understanding-parser-combinators/
Understanding parser combinators: a deep dive - Scott Wlaschin	https://www.youtube.com/watch?v=RDalzi7mhdY

12.1 Monadic parser combinators

12.1.1 Implementing Functor

The implementation of `Functor` should be straight forward: we want to apply a function to the result of a parser.

Listing 12.1.1 of `Parser.hs` (`ParserFunctor`)

Haskell code

```
146 instance Functor Parser where
147     fmap = pMap
```

12.1.2 Implementing Applicative

The goal of `Applicative` is to allow the partial application of functions. In order to use this, we will introduce the `succeed` parser, which always succeeds with a given result:

Listing 12.1.2 of `Parser.hs` (`succeed`)

Haskell code

```
10 succeed :: a -> Parser a
11 succeed a = Parser (\input -> Success a input)
```

This will be used as `pure` to have the function as the parse result without consuming input. Thus, the function is initially injected using `pure` without consuming any input, and then it can be applied to its arguments by chaining parsers that return each argument:

Listing 12.1.3 of `Parser.hs` (`ParserApplicative`)

Haskell code

```
159 instance Applicative Parser where
160     pure = succeed
161     pf <*> pa = (\(f, a) -> f a) <$> andThen pf pa
```

This allows chaining an arbitrary number of parsers and applying a function (that takes the same number of arguments) to their results. For example, we can use `liftA3` to implement `between`:

Listing 12.1.4 of `Parser.hs` (`between'`)

Haskell code

```
165 between' :: Parser a -> Parser b -> Parser c -> Parser c
166 between' pHd pTl p = liftA3 (\_ b _ -> b) pHd p pTl
```

`pHd`, `p` and `pTl` are run sequentially on the input (i.e. each gets the remaining input from the previous one) and the given function is applied to their results.

Applicative also defines `<*>` and `<*`, which allows sequencing parsers and discarding one of the results:

Haskell REPL

```
> runParser (char 'a' *> char 'b') "ab"
Success 'b' ""
> runParser (char 'a' <*> char 'b') "ab"
Success 'a' ""
```

Listing 12.1.5 of Parser.hs (between'')

Haskell code

```

170 | between'' :: Parser a -> Parser b -> Parser c -> Parser c
171 | between'' pHd pTl p = pHd *> (p <*> pTl)

```

Listing 12.1.6 of Parser.hs (date')

Haskell code

```

227 | date' :: Parser Date
228 | date' =
229 |     let sep = char '-'
230 |         nWithDigits :: Int -> Parser Int
231 |         nWithDigits n = read <$> pRepeat n digit
232 |     in Date <$> nWithDigits 4 <*> (sep *> nWithDigits 2) <*> (sep *> nWithDigits 2)

```

Note that in the case of `andThen`, the function that is applied to the results of `liftA2` is the tuple operator `(,)`:

Listing 12.1.7 of Parser.hs (andThen')

Haskell code

```

175 | andThen' :: Parser a -> Parser b -> Parser (a, b)
176 | andThen' pa pb = liftA2 (,) pa pb

```

Finally, in the case of `sequenceA` we know that it takes a `[Parser a]` and returns `Parser [a]`. Thus, if we have `[Parser Char]`, with `sequenceA` we get `Parser [Char]` (i.e. `Parser String`). Using this observation, we can implement the `string` parser by turning the `String` (which is `[Char]`) into a `[Parser Char]` by using `fmap` (i.e. turning each `Char` in the string into a `Parser Char`), then, we use `sequenceA` to run all parsers and collect their results to obtain `Parser [Char]`:

Listing 12.1.8 of Parser.hs (string'')

Haskell code

```

180 | string'' :: String -> Parser String
181 | string'' s = sequenceA (char <$> s)

```

Haskell REPL

```

> runParser (string'' "abc") "abcd"
Success "abc" "d"

```

Exercise 12.1.1

**

Implementing `pRepeat'` using `sequenceA`.

Hints:

You won't need recursion.

12.1.3 Implementing Monad

Knowing that we will implement `Monad` for `Parser`, we can deduce parsing can be interpreted as an action. In the case of parsers, the side-effect is consuming input. Thus, Monads allow using parser combinators without concerning ourselves with how we consume the input. The input is given later and never used directly in the parsers, we just build a parser for input that will be received later. Which is exactly the interpretation given in Lab 10: The side effect for a parser is consuming input, and the input is given later, in order to obtain a result.

Listing 12.1.9 of Parser.hs (ParserMonad)

Haskell code

```

208 instance Monad Parser where
209     pa >>= pb = Parser inner
210     where
211         inner input =
212             case runParser pa input of
213                 Success r rest -> runParser (pb r) rest
214                 Error err -> Error err

```

`pa` is a parser that we need to run in order to obtain the result of type `a`, then we apply `pb` to the result of `pa` to obtain the next parser which will be run on the rest of the input.

`Monad` will allow us to use `do` notation:

Listing 12.1.10 of Parser.hs (between''')

Haskell code

```

218 between''' :: Parser a -> Parser b -> Parser c -> Parser c
219 between''' pHd pTl p = do
220     pHd
221     r <- p
222     pTl
223     return r

```

Compared to the `Applicative` operators (`(*>)`, `<*>`, `<*>`), `do` notation can be ideal to build more complex parsers in a more explicit style:

Listing 12.1.11 of Parser.hs (date''')

Haskell code

```

236 date''' :: Parser Date
237 date''' =
238     let sep = char '-'
239         nWithDigits :: Int -> Parser Int
240         nWithDigits n = read <$> pRepeat n digit
241     in do
242         y <- nWithDigits 4
243         sep
244         m <- nWithDigits 2
245         sep
246         d <- nWithDigits 2
247         return $ Date y m d

```

We should also remember that since a parser is essentially a function that takes a string and returns a structured output, without the input string, we can't, by definition, obtain the output. This further shows that the parser must be executed (by applying it to an input string) in order to obtain the output, as opposed to `Maybe`, where the results can be extracted by using `case`.

12.1.4 Implementing Alternative

The only aspect left is to implement `orElse`, which will be done with `Alternative`.

The Alternative type class

`Alternative` allows handling failure by of an action by trying an alternative. It requires that the data type already implements `Applicative` and requires two functions: `empty` and `<|>`. `empty` returns a result that is considered a failure and `<|>` behaves like `||`: if the action on

the left side succeeds, its result is returned, otherwise the right action is executed and its result is returned.

Listing 12.1.12: Alternative for Maybe

Haskell code

```
instance Alternative Maybe where
  empty = Nothing
  Nothing <|> b = b
  a      <|> _ = a
```

Haskell REPL

```
> Just 1 <|> Just 2
Just 1
> Nothing <|> Just 2
Just 2
```

Implementing Alternative for Parser

To implement `empty`, we introduce a parser that always fails, `pFail`:

Listing 12.1.13 of Parser.hs (`pFail`)

Haskell code

```
15 pFail :: String -> Parser a
16 pFail msg = Parser (\_ -> Error msg)
```

The implementation of `<|>` will be `orElse`:

Listing 12.1.14 of Parser.hs (`ParserAlternative`)

Haskell code

```
190 instance Alternative Parser where
191   empty = pFail ""
192   (<|>) = orElse
```

`Alternative` will also provide `some` and `many`.

`Alternative` also helps to illustrate how parsers can be interpreted as actions with `some` and `many`. Both functions will keep performing an action until it fails, returning the results in a list. Notice how input is consumed each time the parser is run, even though we "run the same parser" (i.e. the input isn't explicitly advanced/consumed in the code).

Haskell REPL

```
> runParser (some (char 'a')) "aaaa"
Success "aaaa" ""
```

Notice how in the case of `Maybe`, since there is no "hidden state", so using `some` or `many` with `Just` will yield an infinite list:

```

> many (Just 1)
Interrupted.
> many (Nothing)
Just []
> some (Just 1)
Interrupted.
> some (Nothing)
Nothing

```

12.2 A JSON parser

Our final example is a simplified JSON parser. We already used JSON in Elm when we received data using a HTTP request.

$\langle json \rangle := \text{'null'} \mid \langle bool \rangle \mid \langle number \rangle \mid \langle string \rangle \mid \langle list \rangle \mid \langle dict \rangle$

$\langle bool \rangle := \text{'true'} \mid \text{'false'}$

$\langle number \rangle := [0'-'9]^+$

$\langle string \rangle := \text{'"} \langle alpha-num \rangle^* \text{'"}$

$\langle alpha-num \rangle := [0'-'9] \mid [a'-'z'] \mid [A'-'Z']$

$\langle list \rangle := \text{'[' ']' } \mid \text{'[' } \langle json \rangle \text{' ,' } \langle json \rangle \text{'* ']'}$

$\langle dict \rangle := \text{'{' '}' } \mid \text{'{' } \langle pair \rangle \text{' ,' } \langle pair \rangle \text{'* '}'}$

$\langle pair \rangle := \langle string \rangle \text{' : ' } \langle json \rangle$

We start with the data definition for `Json`:

Listing 12.2.1 of `Json.hs` (`Json`)

Haskell code

```

7 data Json
8   = JNull
9   | JBool Bool
10  | JInt Int
11  | JString String
12  | JList [Json]
13  | JDict [(String, Json)]
14  deriving (Show)

```

The parsers for $\langle null \rangle$ and $\langle bool \rangle$ are straightforward as they only recognize fixed strings:

Listing 12.2.2 of `Json.hs` (`jNull`)

Haskell code

```

18 jNull :: Parser Json
19 jNull = (const JNull) <$> string "null"

```

Listing 12.2.3 of `Json.hs` (`jBool`)

Haskell code

```

23 jBool :: Parser Json
24 jBool =
25   ((const $ JBool True) <$> string "true")
26   <|> ((const $ JBool False) <$> string "false")

```

```
> runParser jNull "null"
Success JNull ""
> runParser jBool "true"
Success (JBool True) ""
```

The major simplifications are for $\langle number \rangle$ and $\langle string \rangle$:

- `jInt` will only recognize integers (omitting floating point numbers)
- `jString` will only support strings consisting of alpha-numeric characters (omitting support for escape sequences)

Listing 12.2.4 of `Json.hs` (`jInt`)

Haskell code

```
30 | jInt :: Parser Json
31 | jInt = JInt <$> number
```

Listing 12.2.5 of `Json.hs` (`jString`)

Haskell code

```
35 | jString :: Parser Json
36 | jString = JString <$> between' (char '"'') (char '"'') (many alphaNum)
```

Haskell REPL

```
> runParser jInt "12"
Success (JInt 12) ""
> runParser jString "\"abc\""
Success (JString "abc") ""
```

$\langle list \rangle$ and $\langle dict \rangle$ are also fairly straightforward using `between` and `sepBy`:

Listing 12.2.6 of `Json.hs` (`jList`)

Haskell code

```
40 | jList :: Parser Json
41 | jList = JList <$> between' (char '[') (char ']') (sepBy (char ',') json)
```

Listing 12.2.7 of `Json.hs` (`jDict`)

Haskell code

```
45 | jDict :: Parser Json
46 | jDict = JDict <$> between' (char '{') (char '}') (sepBy (char ',') kv)
47 |   where
48 |     kv = (,) <$> k <*> ((char ':') *> v)
49 |     k = between' (char '"'') (char '"'') (many $ satisfies (/= '"''))
50 |     v = json
```

Haskell REPL

```
> runParser jList "[]"
Success (JList []) ""
> runParser jList "[1,2]"
Success (JList [JInt 1,JInt 2]) ""
> runParser jDict "{}"
Success (JDict []) ""
> runParser jDict "{\"a\": []}"
Success (JDict [("a",JList [])]) ""
> runParser jDict "{\"a\":1,\"b\":2}"
Success (JDict [("a",JInt 1),("b",JInt 2)]) ""
```

Finally, `<json>` just uses the previously defined parsers:

Listing 12.2.8 of `Json.hs` (`jjson`)

Haskell code

```
54 | jjson :: Parser Json
55 | jjson = jNull <|> jBool <|> jInt <|> jString <|> jList <|> jDict
```

Haskell REPL

```
> runParser jjson "{\\\"a\\\":null,\\\"b\\\":true,\\\"c\\\":12,\\\"d\\\":[34,true,null,\\\"str\\\"]}"
Success (JDict [
  ("a",JNull),
  ("b",JBool True),
  ("c",JInt 12),
  ("d",JList [JInt 34,JBool True,JNull,JString "str"])
]) ""
```

You might have noticed that in the examples above, we didn't use any whitespace. That is because we didn't handle whitespace at all, so if we tried, the parsing would've failed:

Haskell REPL

```
> runParser jList "[1, 2]"
Error "Unexpected character ','"
> runParser jDict "{\\\"a\\\": []}"
Error "Unexpected character '\\\"'"
```

To handle whitespace, we introduce `<token>`:

Listing 12.2.9 of `Parser.hs` (`token`)

Haskell code

```
257 | token :: Parser a -> Parser a
258 | token = between' ws ws
```

Then use it `<jList>` and `<jDict>` allow whitespace between various separators:

Listing 12.2.10 of `Json.hs` (`jList'`)

Haskell code

```
59 | jList' :: Parser Json
60 | jList' = JList <$> between' lBracket rBracket (sepBy comma jjson)
61 |   where
62 |     lBracket = token $ char '['
63 |     rBracket = token $ char ']'
64 |     comma = token $ char ','
```

Listing 12.2.11 of `Json.hs` (`jDict'`)

Haskell code

```
68 | jDict' :: Parser Json
69 | jDict' = JDict <$> between' lBrace rBrace (sepBy comma kv)
70 |   where
71 |     lBrace = token $ char '{'
72 |     rBrace = token $ char '}'
73 |     comma = token $ char ','
74 |     kv = (,) <$> k <*> ((token $ char ':') *> v)
75 |     k = between' (char '\"') (char '\"') (many $ satisfies (/= '\"'))
76 |     v = jjson
```

Then we just updated `<jjson>` to use `<jList'>` and `<jDict'>`:

```

80 | jjson' :: Parser Jjson
81 | jjson' = jNull <|> jBool <|> jInt <|> jString <|> jList' <|> jDict'

```

```

> runParser jList' "[1, 2]"
Success (JList [JInt 1,JInt 2]) ""
> runParser jDict' "{\"a\": []}"
Success (JDict [(\"a\",JList [])]) ""
> runParser jDict' "{\"a\": 1,\\n\\b\": 2}"
Success (JDict [(\"a\",JInt 1),(\"b\",JInt 2)]) ""

```

Exercise 12.2.1

*

Rewrite the `jList` function using `do` notation, without using `between`.

12.3 Practice problems

Exercise 12.3.1

**

Based on the function stubs and examples in `Html.hs`, implement a parser for HTML.

Subpoint 12.3.1.1

Implement the function `text` function, which parses text, consuming input as long as it doesn't contain the characters `<` or `>`.

Subpoint 12.3.1.2

Implement the function `ident` function, which parses an identifier (starts with a letter, followed by zero or more alphanumeric characters).

Subpoint 12.3.1.3

Implement the `selfClosing` function, which parses a self closing tag. A self closing tag is an identifier followed by a possibly empty list of attributes (see `attributes`) `<` or `/>`.

Subpoint 12.3.1.4

Implement the `openTag` function, which parses an opening tag. A tag is an identifier followed by a possibly empty list of attributes (see `attributes`) `<` or `>`.

Subpoint 12.3.1.5

Implement the `attributes` function, which parses possibly empty list of attributes. An attribute is a key, followed by `=` and a value between double quotes or single quotes. The attributes are separated by whitespace.

Subpoint 12.3.1.6

Implement the `closeTag` function, which parses the given closing tag. A closing tag is an identifier between `</` and `>`.

Subpoint 12.3.1.7

Implement `betweenHtmlTags` function, which runs a parser between html tags, checking that the opening and closing tag match.

Subpoint 12.3.1.8

Implement the `htmlNode` function, which parses a html node (a pair of tags, other tags or text between them, or a self closing tag).

Subpoint 12.3.1.9

Implement the `html` function, which parses a text or html node.